Strings in Java

Sophomore Level: Objects, Efficiency and Data Structures

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. The Design of String
2. equals and hashCode
3. Building Strings Efficiently
4. Ordering Strings
5. Strings in Data Structures
6. Summary

The Design of String

String is an immutable class

```
public final class String
        implements Comparable<String>, CharSequence,
     ... {
    private final byte[] value;  // characters
    private final byte coder;    // LATIN1/UTF16
    private int hash;            // cached
    ...
}
```

(simplified from the JDK 21 source)

How Java makes it immutable:

- `final` class: no subclass can add mutating methods
- `private final` array: nobody outside can reach it
- No method changes `value`; "changing" methods build a new `String`
- Constructors copy the given `char[]`, so changing your array later does not affect the string

Same recipe for your classes

`final` class, `private final` fields, no setters, defensive copies.

Literals, the pool and `intern()`

codeoutput/Pool.java — Java

```java
public class Pool {
    public static void main(String[] args) {
        String a = "java";
        String b = "java";
        String c = new String("java");
        String d = "ja" + "va";          // constant
        String half = "ja";
        String e = half + "va";          // computed at run time
        System.out.println("a == b: " + (a == b));
        System.out.println("a == c: " + (a == c));
        System.out.println("a == d: " + (a == d));
        System.out.println("a == e: " + (a == e));
        System.out.println("intern: " + (a == e.intern()));
        System.out.println("a.equals(e): " + a.equals(e));
    }
}
```

`"ja" + "va"` is a **compile-time constant**: `javac` writes the literal `"java"` into the class file. `half + "va"` is computed at run time and gives a new object. `intern()` returns the pooled one.

Literals, the pool and `intern()`

codeoutput/Pool.java — Java

```java
public class Pool {
    public static void main(String[] args) {
        String a = "java";
        String b = "java";
        String c = new String("java");
        String d = "ja" + "va";           // constant
        String half = "ja";
        String e = half + "va";           // computed at run time
        System.out.println("a == b: " + (a == b));
        System.out.println("a == c: " + (a == c));
        System.out.println("a == d: " + (a == d));
        System.out.println("a == e: " + (a == e));
        System.out.println("intern: " + (a == e.intern()));
        System.out.println("a.equals(e): " + a.equals(e));
    }
}
```

Output codeoutput/Pool.txt

```
a == b: true
a == c: false
a == d: true
a == e: false
intern: true
a.equals(e): true
```

`"ja" + "va"` is a **compile-time constant**: `javac` writes the literal `"java"` into the class file. `half + "va"` is computed at run time and gives a new object. `intern()` returns the pooled one.

equals and hashCode

The equals/hashCode contract

The contract (`java.lang.Object`)

1. `a.equals(b)` ⇒ `a.hashCode() == b.hashCode()`
2. The converse need **not** hold: different strings may share a hash (a **collision**)
3. `hashCode` must not change while the object is in a hash table

`String` satisfies all three; immutability gives (3) for free.

`String.hashCode()` for $s = s_0 s_1 \dots s_{n-1}$:

$$h(s) = \sum_{i=0}^{n-1} s_i \cdot 31^{n-1-i} \pmod{2^{32}}$$

computed by Horner's rule: $h \leftarrow 31h + s_i$.

Example: $h(\texttt{"Aa"}) = 65 \cdot 31 + 97 = 2112$ and $h(\texttt{"BB"}) = 66 \cdot 31 + 66 = 2112$: a collision.

Computing hashCode by hand

codeoutput/HashCodes.java — Java

```java
public class HashCodes {
    static int hash(String s) {          // = String.hashCode()
        int h = 0;
        for (int i = 0; i < s.length(); i++) {
            h = 31 * h + s.charAt(i);
        }
        return h;
    }

    public static void main(String[] args) {
        String[] words = {"a", "ab", "java", "Aa", "BB", "
     Yeditepe"};
        for (String s : words) {
            int h1 = s.hashCode(), h2 = hash(s);
            System.out.printf("%-9s %12d %12d%n", s, h1, h2);
        }
    }
}
```

Our `hash` matches `String.hashCode()`. Overflow wraps around (`Yeditepe` is negative). `String` caches its hash in a field.

Computing hashCode by hand

codeoutput/HashCodes.java — Java

```java
public class HashCodes {
    static int hash(String s) {          // = String.hashCode()
        int h = 0;
        for (int i = 0; i < s.length(); i++) {
            h = 31 * h + s.charAt(i);
        }
        return h;
    }

    public static void main(String[] args) {
        String[] words = {"a", "ab", "java", "Aa", "BB", "
     Yeditepe"};
        for (String s : words) {
            int h1 = s.hashCode(), h2 = hash(s);
            System.out.printf("%-9s %12d %12d%n", s, h1, h2);
        }
    }
}
```

Output
codeoutput/HashCodes.txt

a	97	97
ab	3105	3105
java	3254818	3254818
Aa	2112	2112
BB	2112	2112
Yeditepe	-1396670377	-1396670377

Our hash matches `String.hashCode()`. Overflow wraps around (`Yeditepe` is negative). `String` caches its hash in a field.

Why keys should be immutable

codeoutput/KeyTrap.java — Java

```java
import java.util.HashSet;
import java.util.Set;

public class KeyTrap {
    public static void main(String[] args) {
        Set<String> names = new HashSet<>();
        names.add("ali");
        String typed = new String("ali");    // another object
        System.out.println(names.contains(typed));
        Set<StringBuilder> builders = new HashSet<>();
        builders.add(new StringBuilder("ali"));
        StringBuilder probe = new StringBuilder("ali");
        System.out.println(builders.contains(probe));
        StringBuilder key = new StringBuilder("ali");
        builders.add(key);
        key.append("ce");          // changes inside the set!
        System.out.println(builders.contains(key));
    }
}
```

StringBuilder has no equals/hashCode of its own and can change: use String keys.

Why keys should be immutable

codeoutput/KeyTrap.java — Java

```java
import java.util.HashSet;
import java.util.Set;

public class KeyTrap {
    public static void main(String[] args) {
        Set<String> names = new HashSet<>();
        names.add("ali");
        String typed = new String("ali");    // another object
        System.out.println(names.contains(typed));
        Set<StringBuilder> builders = new HashSet<>();
        builders.add(new StringBuilder("ali"));
        StringBuilder probe = new StringBuilder("ali");
        System.out.println(builders.contains(probe));
        StringBuilder key = new StringBuilder("ali");
        builders.add(key);
        key.append("ce");          // changes inside the set!
        System.out.println(builders.contains(key));
    }
}
```

Output
codeoutput/KeyTrap.txt

```
true
false
true
```

StringBuilder has no equals/hashCode of its own and can change: use String keys.

Building Strings Efficiently

+= in a loop versus StringBuilder

codeoutput/Timing.java — Java

```java
public class Timing {
    static String plus(int n) {
        String s = "";
        for (int i = 0; i < n; i++) s += "x";        // copies s each time
        return s;
    }
    static String builder(int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append("x");  // amortized O(1)
        return sb.toString();
    }
    public static void main(String[] args) {
        plus(20_000); builder(20_000);                // warm up the JIT
        for (int n = 20_000; n <= 160_000; n *= 2) {
            long t0 = System.nanoTime();
            String a = plus(n);
            long t1 = System.nanoTime();
            String b = builder(n);
            long t2 = System.nanoTime();
            System.out.printf("n=%6d  +=%7.1f ms  builder=%5.2f ms  same=%b%n",
                    n, (t1 - t0) / 1e6, (t2 - t1) / 1e6, a.equals(b));
        }
    }
}
```

One run; your numbers will differ. Doubling n: += about 4× slower (quadratic), builder near 1 ms.

+= in a loop versus StringBuilder

codeoutput/Timing.java — Java

```java
public class Timing {
    static String plus(int n) {
        String s = "";
        for (int i = 0; i < n; i++) s += "x";        // copies s each time
        return s;
    }
    static String builder(int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) sb.append("x");  // amortized O(1)
        return sb.toString();
    }
    public static void main(String[] args) {
        plus(20_000); builder(20_000);                // warm up the JIT
        for (int n = 20_000; n <= 160_000; n *= 2) {
            long t0 = System.nanoTime();
            String a = plus(n);
            long t1 = System.nanoTime();
            String b = builder(n);
            long t2 = System.nanoTime();
            System.out.printf("n=%6d  +=%7.1f ms  builder=%5.2f ms  same=%b%n",
                    n, (t1 - t0) / 1e6, (t2 - t1) / 1e6, a.equals(b));
        }
    }
}
```

Output — codeoutput/Timing.txt

```
n= 20000  +=   115.7 ms  builder= 0.97 ms  same=true
n= 40000  +=   307.6 ms  builder= 1.64 ms  same=true
n= 80000  +=   937.4 ms  builder= 1.68 ms  same=true
n=160000  += 3445.8 ms  builder= 0.86 ms  same=true
```

One run; your numbers will differ. Doubling n: += about $4\times$ slower (quadratic), builder near 1 ms.

Why the builder is fast: doubling

codeoutput/Capacity.java **Java**

```java
public class Capacity {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        int last = sb.capacity();
        System.out.println("length   0  capacity " + last);
        for (int i = 1; i <= 600; i++) {
            sb.append('x');
            if (sb.capacity() != last) {
                last = sb.capacity();
                System.out.printf("length %3d  capacity %d%n",
                                  i, last);
            }
        }
    }
}
```

When full, the capacity c grows to $2c + 2$. Appending n characters copies fewer than $n + \frac{n}{2} + \frac{n}{4} + \cdots < 2n$ characters in total: each append is **amortized** $O(1)$. With +=, step k copies k characters: $O(n^2)$ in total.

Why the builder is fast: doubling

codeoutput/Capacity.java — Java

```java
public class Capacity {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        int last = sb.capacity();
        System.out.println("length   0  capacity " + last);
        for (int i = 1; i <= 600; i++) {
            sb.append('x');
            if (sb.capacity() != last) {
                last = sb.capacity();
                System.out.printf("length %3d  capacity %d%n",
                                  i, last);
            }
        }
    }
}
```

Output
codeoutput/Capacity.txt

```
length   0  capacity 16
length  17  capacity 34
length  35  capacity 70
length  71  capacity 142
length 143  capacity 286
length 287  capacity 574
length 575  capacity 1150
```

When full, the capacity c grows to $2c + 2$. Appending n characters copies fewer than $n + \frac{n}{2} + \frac{n}{4} + \cdots < 2n$ characters in total: each append is **amortized** $O(1)$. With +=, step k copies k characters: $O(n^2)$ in total.

Ordering Strings

Comparable and Comparator

Natural order: `String` implements `Comparable<String>`

- `compareTo` compares `char` codes (UTF-16 values)
- `Arrays.sort`, `Collections.sort`, `TreeMap` use it by default

Other orders: pass a `Comparator<String>`

- `String.CASE_INSENSITIVE_ORDER`
- `Comparator.comparing(String::length)`
- `Collator.getInstance(locale)`: a language's alphabet
- combine with `.thenComparing(...)`, `.reversed()`

The Turkish alphabet:

a b c ç d e f g ğ h ı i j k l m n o ö p r s ş t u ü v y z

In Unicode, ç (231), ı (305), ü (252) all come after z (122), so `compareTo` puts çay after zeytin.

Rule

Sort for **people** with a `Collator`; sort for **programs** (keys, IDs) with `compareTo`.

Four ways to sort the same words

codeoutput/SortStrings.java — Java

```java
import java.text.Collator;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Locale;

public class SortStrings {
    public static void main(String[] args) {
        String[] a = {"zeytin", "Elma", "çay", "armut",
                      "ılık", "incir", "Üzüm"};
        Arrays.sort(a);                                  // char codes
        System.out.println(Arrays.toString(a));
        Arrays.sort(a, String.CASE_INSENSITIVE_ORDER);
        System.out.println(Arrays.toString(a));
        Collator tr = Collator.getInstance(Locale.forLanguageTag("tr"));
        Arrays.sort(a, tr);                              // Turkish alphabet
        System.out.println(Arrays.toString(a));
        Arrays.sort(a, Comparator.comparing(String::length)
                                 .thenComparing(tr));    // length, then tr
        System.out.println(Arrays.toString(a));
    }
}
```

Only the `Collator` gives the dictionary order: `armut, çay, Elma, ılık, incir, Üzüm, zeytin`.

Four ways to sort the same words

codeoutput/SortStrings.java — Java

```java
import java.text.Collator;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Locale;

public class SortStrings {
    public static void main(String[] args) {
        String[] a = {"zeytin", "Elma", "çay", "armut",
                      "ılık", "incir", "Üzüm"};
        Arrays.sort(a);                                  // char codes
        System.out.println(Arrays.toString(a));
        Arrays.sort(a, String.CASE_INSENSITIVE_ORDER);
        System.out.println(Arrays.toString(a));
        Collator tr = Collator.getInstance(Locale.forLanguageTag("tr"));
        Arrays.sort(a, tr);                              // Turkish alphabet
        System.out.println(Arrays.toString(a));
        Arrays.sort(a, Comparator.comparing(String::length)
                                 .thenComparing(tr));    // length, then tr
        System.out.println(Arrays.toString(a));
    }
}
```

Output — codeoutput/SortStrings.txt

```
[Elma, armut, incir, zeytin, Üzüm, çay, ılık]
[armut, Elma, ılık, incir, zeytin, çay, Üzüm]
[armut, çay, Elma, ılık, incir, Üzüm, zeytin]
[çay, Elma, ılık, Üzüm, armut, incir, zeytin]
```

Only the `Collator` gives the dictionary order: `armut, çay, Elma, ılık, incir, Üzüm, zeytin`.

Strings in Data Structures

Word frequencies with a TreeMap

codeoutput/WordFreq.java — Java

```java
import java.util.Map;
import java.util.TreeMap;

public class WordFreq {
    public static void main(String[] args) {
        String text = "The cat and the hat; the cat sat. And then?";
        Map<String, Integer> freq = new TreeMap<>();      // sorted keys
        for (String w : text.toLowerCase().split("[^\\p{L}]+")) {
            if (!w.isEmpty()) {
                freq.merge(w, 1, Integer::sum);           // +1
            }
        }
        System.out.println(freq);
        String top = null;
        for (String w : freq.keySet()) {
            if (top == null || freq.get(w) > freq.get(top)) {
                top = w;
            }
        }
        System.out.println("most frequent: " + top + " x" + freq.get(top));
    }
}
```

`[^\\p{L}]+` splits at anything that is not a **letter** in any language (`\\W+` would also split Turkish words at `ç, ş, ı`). `merge(w, 1, Integer::sum)` adds 1, starting from 1.

Word frequencies with a TreeMap

codeoutput/WordFreq.java — Java

```java
import java.util.Map;
import java.util.TreeMap;

public class WordFreq {
    public static void main(String[] args) {
        String text = "The cat and the hat; the cat sat. And then?";
        Map<String, Integer> freq = new TreeMap<>();      // sorted keys
        for (String w : text.toLowerCase().split("[^\\p{L}]+")) {
            if (!w.isEmpty()) {
                freq.merge(w, 1, Integer::sum);           // +1
            }
        }
        System.out.println(freq);
        String top = null;
        for (String w : freq.keySet()) {
            if (top == null || freq.get(w) > freq.get(top)) {
                top = w;
            }
        }
        System.out.println("most frequent: " + top + " x" + freq.get(top));
    }
}
```

Output codeoutput/WordFreq.txt

```
{and=2, cat=2, hat=1, sat=1, the=3,
       then=1}
most frequent: the x3
```

`[^\\p{L}]+` splits at anything that is not a **letter** in any language (`\\W+` would also split Turkish words at ç, ş, ı). `merge(w, 1, Integer::sum)` adds 1, starting from 1.

Anagrams: sorting or counting

codeoutput/Anagram.java — Java

```java
import java.util.Arrays;
public class Anagram {
    static boolean bySorting(String a, String b) {   // O(n log n)
        char[] x = a.toCharArray(), y = b.toCharArray();
        Arrays.sort(x);
        Arrays.sort(y);
        return Arrays.equals(x, y);
    }
    static boolean byCounting(String a, String b) {  // O(n), a-z only
        if (a.length() != b.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < a.length(); i++) {
            count[a.charAt(i) - 'a']++;
            count[b.charAt(i) - 'a']--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        String[][] pairs = {{"listen", "silent"}, {"evil", "vile"}, {"java", "jaguar"}};
        for (String[] p : pairs)
            System.out.println(p[0] + " " + p[1] + ": " + bySorting(p[0], p[1])
                               + " " + byCounting(p[0], p[1]));
    }
}
```

`c - 'a'` is 0..25 because `'a'..'z'` are consecutive; for other alphabets count in a `HashMap`.

Anagrams: sorting or counting

codeoutput/Anagram.java — Java

```java
import java.util.Arrays;
public class Anagram {
    static boolean bySorting(String a, String b) {   // O(n log n)
        char[] x = a.toCharArray(), y = b.toCharArray();
        Arrays.sort(x);
        Arrays.sort(y);
        return Arrays.equals(x, y);
    }
    static boolean byCounting(String a, String b) {  // O(n), a-z only
        if (a.length() != b.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < a.length(); i++) {
            count[a.charAt(i) - 'a']++;
            count[b.charAt(i) - 'a']--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        String[][] pairs = {{"listen", "silent"}, {"evil", "vile"}, {"java", "jaguar"}};
        for (String[] p : pairs)
            System.out.println(p[0] + " " + p[1] + ": " + bySorting(p[0], p[1])
                               + " " + byCounting(p[0], p[1]));
    }
}
```

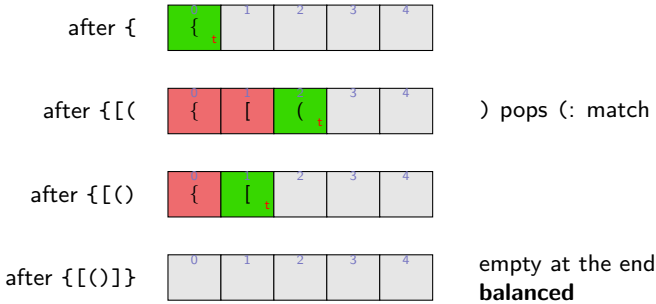
Output
codeoutput/Anagram.txt

```
listen silent: true true
evil vile: true true
java jaguar: false false
```

c - 'a' is 0..25 because 'a'..'z' are consecutive; for other alphabets count in a `HashMap`.

Matching brackets with a stack

Read `{[()]}` from left to right: **push** an opening bracket, **pop** on a closing one and check that they match.



The green box is the **top** (`t`). Drawn with `\hStack` from `hblibDatastructures`.

Bracket matching in Java

codeoutput/Brackets.java — Java

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class Brackets {
    static boolean balanced(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if ("([{".indexOf(c) >= 0) {
                stack.push(c);                          // opening: push
            } else if (")]}".indexOf(c) >= 0) {
                if (stack.isEmpty()) return false;      // nothing to close
                char open = stack.pop();                // closing: pop
                if ("([{".indexOf(open) != ")]}".indexOf(c)) return false;
            }
        }
        return stack.isEmpty();                         // all closed?
    }

    public static void main(String[] args) {
        String[] tests = {"{[()]}", "a(b[c]d)e", "([)]", "((", "))"};
        for (String s : tests) {
            System.out.println(s + " -> " + balanced(s));
        }
    }
}
```

`ArrayDeque` is Java's recommended stack. Each character is pushed and popped at most once: $O(n)$.

Bracket matching in Java

codeoutput/Brackets.java — Java

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class Brackets {
    static boolean balanced(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if ("([{".indexOf(c) >= 0) {
                stack.push(c);                          // opening: push
            } else if (")]}".indexOf(c) >= 0) {
                if (stack.isEmpty()) return false;      // nothing to close
                char open = stack.pop();                // closing: pop
                if ("([{".indexOf(open) != ")]}".indexOf(c)) return false;
            }
        }
        return stack.isEmpty();                         // all closed?
    }

    public static void main(String[] args) {
        String[] tests = {"{[()]}", "a(b[c]d)e", "([)]", "((", "))"};
        for (String s : tests) {
            System.out.println(s + " -> " + balanced(s));
        }
    }
}
```

Output codeoutput/Brackets.txt

```
{[()]} -> true
a(b[c]d)e -> true
([)] -> false
(( -> false
)) -> false
```

`ArrayDeque` is Java's recommended stack. Each character is pushed and popped at most once: $O(n)$.

Summary

Costs of common operations

n = `s.length()`, m = length of the other string, k = length of the result

Operation	Time	Note
`length()`, `charAt(i)`	$O(1)$	array access
`substring(b, e)`	$O(k)$	copies $k = e - b$ chars (since Java 7)
`a + b`, `concat`	$O(n + m)$	new array
`equals`, `compareTo`	$O(\min(n, m))$	stops at the first difference
`hashCode()`	$O(n)$ once	then cached
`indexOf(t)`	$O(n \cdot m)$ worst	naive search (see the junior slides)
`sb.append(x)`	$O(1)$ amortized	capacity doubling
`s += x` in a loop of n	$O(n^2)$	copies everything each time

Exercises

1. Write an immutable class `Name(first, last)` with correct `equals`, `hashCode` and `compareTo` (last name, then first name, with a Turkish `Collator`).
2. Generate 8 different strings with the same `hashCode` (hint: `"Aa"` and `"BB"` combine).
3. Group words into anagram classes with a `Map<String, List<String>>` keyed by the sorted letters.
4. Extend `Brackets` to ignore brackets inside quotes: `"a(\"b\")"`.
5. Measure `String.join`, `StringBuilder` and `+=` for 100 000 short words; plot the times.
6. Implement `reverseWords("the cat sat")` $\rightarrow$ `"sat cat the"` in $O(n)$ with a `StringBuilder`.

Questions?